

```

import requests
import os
import time
from datetime import datetime
from dotenv import load_dotenv

# ----- LOAD ENV -----
load_dotenv()
API_KEY = os.getenv("OPENROUTER_API_KEY")
MODEL = "openai/gpt-3.5-turbo"

timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
OUTPUT_FILE = f"output_{timestamp}.txt"

# ----- LOG -----
def log(text):
    with open(OUTPUT_FILE, "a", encoding="utf-8") as f:
        f.write(text + "\n")

def log_and_print(text):
    print(text)
    log(text)

def log_input(label, value):
    log(f"[INPUT] {label}: {value}")

# ----- LLM -----
def call_llm(prompt):
    url = "https://openrouter.ai/api/v1/chat/completions"

    headers = {
        "Authorization": f"Bearer {API_KEY}",
        "Content-Type": "application/json"
    }

    data = {
        "model": MODEL,
        "messages": [{"role": "user", "content": prompt}]
    }

    for _ in range(2):
        start = time.time()
        try:
            r = requests.post(url, headers=headers, json=data, timeout=10)
            if r.status_code == 200:
                return r.json()["choices"][0]["message"]["content"], time.time() - start
        except:
            pass

```

```
return " 🚨 API failed", 0
```

```
# ----- OPERATORS -----
```

```
NUM_TO_OP = {  
    "1": "+", "2": "x", "3": "-",  
    "4": "⊕", "5": "^", "6": "□", "7": "!"  
}
```

```
VALID_OPS = ["+", "x", "-", "⊕", "^", "□", "!" ]
```

```
OP_MEANING = {  
    "+": "combine",  
    "x": "synergy",  
    "-": "remove",  
    "⊕": "evolve",  
    "^": "amplify",  
    "□": "practical",  
    "!": "disrupt"  
}
```

```
# ----- VISUAL MODES -----
```

```
MODES = {  
    "1": (" 🧠 Balanced", ["⊕", "^", "!"]),  
    "2": (" 💡 Innovator", ["⊕", "^", "!", "!"]),  
    "3": (" 🛠 Builder", ["□", "^", "⊕"]),  
    "4": (" 🔥 Disruptor", ["!", "⊕", "^"]),  
    "5": (" 🌀 Chaos", ["!", "!", "!"]),  
    "6": (" 🙏 Philosopher", ["+", "-", "⊕"])  
}
```

```
# ----- LEGEND -----
```

```
def print_legend():  
    print("\n--- OPERATORS ---")  
    for k,v in NUM_TO_OP.items():  
        print(f'{k} → {v} ({OP_MEANING[v]})')  
    print("Repeat = strength (777 = strong)")  
    print("-----\n")
```

```
def print_modes():  
    print("\n--- VISUAL MODES ---")  
    for k, v in MODES.items():  
        name = v[0]  
        ops = v[1]  
        ops_str = " ".join(ops)  
        meanings = " | ".join([OP_MEANING[o[0]] for o in ops])  
  
        print(f'{k}: {name} → ({ops_str})')
```

```

        print(f" {meanings}\n")

# ----- PARSE -----
def convert_numeric_ops(s):
    result = []
    for token in s.split(","):
        token = token.strip()
        if not token:
            continue
        base = token[0]
        if base in NUM_TO_OP:
            result.append(NUM_TO_OP[base] * len(token))
        else:
            result.append("auto")
    return result

def validate_ops(ops, iterations):
    if len(ops) > iterations:
        ops = ops[:iterations]
    return ops

# ----- APPLY -----
def apply_operator(op, idea, seed):
    base = f'Stay aligned with "{seed}". Be sharp.\n'

    if op[0] == "!":
        return call_llm(base + f'Challenge assumptions:\n{idea}')
    if op[0] == "⊖":
        return call_llm(base + f'Evolve idea:\n{idea}')
    if op[0] == "^":
        return call_llm(base + f'Amplify insight:\n{idea}')
    if op[0] == "□":
        return call_llm(base + f'Make practical:\n{idea}')
    if op[0] == "-":
        return call_llm(base + f'Remove weak parts:\n{idea}')
    if op[0] == "+":
        return call_llm(base + f'Combine ideas:\n{idea}')

    return idea, 0

# ----- AUTO -----
def auto_operator(current):
    r, _ = call_llm(f'Pick best operator:\n{current}')
    return r.strip()

# ----- SCORE -----
def score_step(idea):
    r, _ = call_llm(f'')

```

Rate idea:

Novelty: X

Usefulness: X

Feasibility: X

Idea:

{idea}

""")

```
scores = {"Novelty":5,"Usefulness":5,"Feasibility":5}
```

```
for line in r.split("\n"):
```

```
    if ":" in line:
```

```
        try:
```

```
            k,v = line.split(":")
```

```
            scores[k.strip()] = int(v.strip())
```

```
        except:
```

```
            pass
```

```
return scores, sum(scores.values())
```

```
# ----- SUGGEST -----
```

```
def suggest_operator(idea):
```

```
    r, _ = call_llm(f"Suggest next operator:\n{idea}")
```

```
    return r.strip()
```

```
# ----- MAIN -----
```

```
def run():
```

```
    total_start = time.time()
```

```
    log("==== SESSION =====")
```

```
    # THINKING MODE
```

```
    print("\n1 FAST | 2 RECURSIVE")
```

```
    thinking = input("Thinking mode: ").strip()
```

```
    seed = input("Enter seed: ").strip()
```

```
    log_input("seed", seed)
```

```
    # OPERATOR MODE
```

```
    print("\n1 Visual | 2 Manual | 3 Hybrid")
```

```
    choice = input("Operator mode: ").strip()
```

```
    mode_ops = ["⌚", "^", "!"]
```

```
    if choice == "1":
```

```
        print_modes()
```

```

m = input("Choose preset: ").strip()
mode_ops = MODES.get(m, mode_ops)[1]

print("\nSelected:", " ".join(mode_ops))
edit = input("Edit operators? (optional): ").strip()
if edit:
    mode_ops = convert_numeric_ops(edit)

iterations = len(mode_ops)

elif choice == "2":
    print_legend()
    ops_input = input("Operators: ")
    mode_ops = convert_numeric_ops(ops_input)
    iterations = int(input("Iterations: ") or 3)

else:
    print_modes()
    m = input("Base preset: ").strip()
    base = MODES.get(m, mode_ops)[1]

    print("\nSelected:", " ".join(base))

    print_legend()
    tweak = input("Add operators: ")
    mode_ops = base + convert_numeric_ops(tweak)

    iterations = len(mode_ops)

mode_ops = validate_ops(mode_ops, iterations)

log_and_print(f"\n=== SEED ===\n{seed}")
log_and_print("\nOPERATOR PATH:")
log_and_print(" ".join(mode_ops))

current = seed
best_score = -1
best_idea = ""
total_model_time = 0

# ----- EXECUTION -----
if thinking == "1":
    result, t = call_llm(f"Improve idea:\n{seed}")
    log_and_print("\n ⚡ FAST RESULT:\n" + result)
    total_model_time += t

else:
    for i in range(iterations):

```

```

step_start = time.time()

op = mode_ops[i] if i < len(mode_ops) else auto_operator(current)

log_and_print(f"\nSTEP {i+1} OPERATOR: {op}")

current, t = apply_operator(op, current, seed)
total_model_time += t

scores, total = score_step(current)

step_time = time.time() - step_start

log_and_print(f"\nRESULT:\n{current}")
log_and_print(f"SCORE: {scores} TOTAL={total}")
log_and_print(f"STEP TIME: {round(step_time,2)}s | MODEL: {round(t,2)}s")

suggestion = suggest_operator(current)
log_and_print(f"NEXT: {suggestion}")

if total > best_score:
    best_score = total
    best_idea = current

log_and_print("\n--- FINAL ---")

final, t = call_llm(f"Summarize:\n{best_idea}")
total_model_time += t

log_and_print("\n🌟 BEST:")
log_and_print(final)

total_time = time.time() - total_start

log_and_print(f"\n🕒 MODEL TOTAL: {round(total_model_time,2)}s")
log_and_print(f"🕒 TOTAL TIME: {round(total_time,2)}s")

print(f"\nSaved to {OUTPUT_FILE}")

# ----- ENTRY -----
if __name__ == "__main__":
    run()

```